

File Read and Write Operations in Python

Introduction

Files are used to store data permanently on a computer. Unlike variables, whose data is lost when a program ends, files allow information to be saved and retrieved later.

Python provides built-in functions to create, read, write, update, and manage files easily.

File handling is an essential skill for developing real-world applications such as:

- Student Management Systems
 - Banking Applications
 - Inventory Systems
 - Quiz Applications
 - Attendance Systems
 - Data Analysis Projects
-

What is a File?

A **file** is a collection of data stored on a storage device.

Examples:

- students.txt
- data.csv
- report.pdf
- image.png

Python commonly works with text files (.txt) for beginners.

Why Use Files?

Without files:

```
name = "Rahul"
```

The value is lost when the program closes.

With files:

```
Rahul
```

The data remains stored permanently.

Advantages

- ✓ Permanent Storage
 - ✓ Easy Data Retrieval
 - ✓ Large Data Management
 - ✓ Sharing Information
 - ✓ Maintaining Records
-

File Handling Process

File handling generally follows three steps:

Open File



Read / Write Data



Close File

Example:

```
file = open("data.txt", "r")
```

Operations

```
file.close()
```

Opening a File

Syntax

```
open(filename, mode)
```

Example:

```
file = open("student.txt", "r")
```

File Modes

Mode Description

"r" Read Mode

"w" Write Mode

"a" Append Mode

Mode Description

"x" Create New File

"r+" Read and Write

"w+" Write and Read

"a+" Append and Read

"rb" Read Binary

"wb" Write Binary

Read Mode (r)

Used to read existing files.

Example

Suppose student.txt contains:

John

Rahul

Priya

Python Code:

```
file = open("student.txt", "r")
```

```
content = file.read()
```

```
print(content)
```

```
file.close()
```

Output:

John

Rahul

Priya

Reading a File

1. read()

Reads the entire file.

```
file = open("data.txt", "r")
```

```
data = file.read()
```

```
print(data)
```

```
file.close()
```

2. readline()

Reads one line at a time.

```
file = open("data.txt", "r")
```

```
print(file.readline())
```

```
print(file.readline())
```

```
file.close()
```

3. readlines()

Reads all lines into a list.

```
file = open("data.txt", "r")
```

```
lines = file.readlines()
```

```
print(lines)
```

```
file.close()
```

Output:

```
['John\n', 'Rahul\n', 'Priya\n']
```

Writing to a File

Write Mode (w)

Creates a new file or overwrites an existing file.

Example

```
file = open("data.txt", "w")
```

```
file.write("Hello Python")
```

```
file.close()
```

Content:

Hello Python

Important

If the file already contains data:

Old Data

After writing:

```
file = open("data.txt", "w")  
file.write("New Data")
```

The old content is deleted.

New file content:

New Data

Writing Multiple Lines

```
file = open("students.txt", "w")
```

```
file.write("Rahul\n")  
file.write("Priya\n")  
file.write("Amit\n")
```

```
file.close()
```

Output File:

Rahul
Priya
Amit

Append Mode (a)

Adds data to the end of the file without deleting existing content.

Example

Current file:

Rahul

Priya

Code:

```
file = open("students.txt", "a")
```

```
file.write("\nAmit")
```

```
file.close()
```

Updated File:

Rahul

Priya

Amit

Creating a New File

Using x mode.

```
file = open("newfile.txt", "x")
```

```
file.close()
```

If the file already exists, Python generates an error.

Using with Statement

The best practice for file handling.

Advantages

- Automatically closes files.
- Cleaner code.
- Prevents resource leaks.

Example

```
with open("data.txt", "r") as file:  
    content = file.read()
```

```
print(content)
```

No need to write:

```
file.close()
```

Reading File Using Loop

with open("students.txt", "r") as file:

```
    for line in file:  
        print(line.strip())
```

Output:

Rahul
Priya
Amit

File Exists Check

Using the os module.

```
import os  
  
if os.path.exists("data.txt"):  
    print("File exists")  
else:  
    print("File not found")
```

Deleting a File

```
import os  
  
os.remove("data.txt")
```

Working with User Input

Save User Data

```
name = input("Enter name: ")  
  
with open("users.txt", "a") as file:  
    file.write(name + "\n")
```

Example 1: Student Record System

```
student_name = input("Enter Student Name: ")
```

```
with open("students.txt", "a") as file:
    file.write(student_name + "\n")

print("Student saved successfully.")
```

Example 2: Read Student Records

```
with open("students.txt", "r") as file:

    print("Student Records:\n")

    print(file.read())
```

Example 3: Attendance System

```
name = input("Enter your name: ")

with open("attendance.txt", "a") as file:
    file.write(name + "\n")

print("Attendance Marked")
```

Example 4: Quiz Score Storage

```
score = 18

with open("scores.txt", "a") as file:
    file.write(f"Score: {score}\n")
```

Output File:

```
Score: 18
Score: 15
Score: 20
```

Exception Handling in File Operations

Sometimes a file may not exist.

Example

try:

```
file = open("data.txt", "r")
```



```
print(file.read())
```

```
file.close()
```

except FileNotFoundError:

```
    print("File not found.")
```

Output:

File not found.

Common Errors

File Not Found

```
open("abc.txt", "r")
```

Error:

FileNotFoundError

Permission Error

Occurs when Python doesn't have permission to access a file.

Invalid Mode

```
open("data.txt", "z")
```

Error:

ValueError

Mini Project: Notes Saver

Program

```
while True:
```

```
    note = input("Enter Note (type exit to stop): ")
```

```
    if note.lower() == "exit":
        break
```

```
with open("notes.txt", "a") as file:
    file.write(note + "\n")

print("Notes Saved Successfully")
```

Mini Project: Student Database

```
while True:

    print("\n1. Add Student")
    print("2. View Students")
    print("3. Exit")

    choice = input("Enter choice: ")

    if choice == "1":

        name = input("Student Name: ")

        with open("students.txt", "a") as file:
            file.write(name + "\n")

    elif choice == "2":

        with open("students.txt", "r") as file:
            print(file.read())

    elif choice == "3":
        break
```

Best Practices

Use with

```
with open("data.txt", "r") as file:
    data = file.read()
```

Handle Exceptions

```
try:
    with open("data.txt", "r") as file:
        print(file.read())
except:
    print("Error")
```

Close Files Properly

Always close files or use with.

Use Meaningful File Names

students.txt

attendance.txt

scores.txt

Learning Outcomes

After completing this topic, students will be able to:

- ✓ Create files
 - ✓ Read file contents
 - ✓ Write data to files
 - ✓ Append new information
 - ✓ Handle file-related errors
 - ✓ Store user data permanently
 - ✓ Build record management systems
 - ✓ Develop real-world Python applications
-

Summary

File handling allows Python programs to store and retrieve data permanently. The most commonly used operations are:

- open()
- read()
- readline()
- readlines()
- write()
- append()
- close()

Using file operations, developers can build applications such as student management systems, attendance trackers, quiz applications, note-taking tools, and many other real-

world software solutions. Mastering file handling is a crucial step toward becoming a proficient Python programmer.